

Number: P3449R0
Date: 2024-10-15
Audience: LEWG
Reply-to: [Hana.Dusíková](#)

Table of contents

- [Why would you want to do this?](#)
- [Simple example](#)
- [Implementation experience](#)
- [Impact on existing code](#)
- [Intention for wording changes](#)
- [Proposed changes to wording](#)
 - [Feature test macros](#)

constexpr std::generator

This paper is proposing making `std::generator` constexpr and it depends on [P3367: constexpr coroutines](#). I was told to put it into a separate paper, so here it is. I don't want this only coroutine type to lag behind language as before.

Why would you want to do this?

Well, you just told me coroutines are the best way to solve some problems, so wouldn't I also want to use the Best Way at compile time? (quote from Jason Turner, co-author of *"constexpr all the things"* talk)

Simple example

When this paper is merged into the standard, users will be able to use `std::generator`-like coroutines to generate or calculate data.

```
template <typename T> constexpr auto fib
{
    T a = 0;
    T b = 1;
    co_yield a;
    do {
        co_yield b;
        auto tmp = b;
        b += a;
        a = tmp;
    } while (true);
}

template <typename T, size_t N> constexpr
auto res = std::array<T, N>{};
std::ranges::copy(fib<T>() | std::views::take(N), res);

constexpr auto cached_fibonacci = calculate_fibonacci;
```

Implementation